Q1：Does it detail the differing results caused by using different embedding models?

- Model A：sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2

  ```
  📊 FINAL SCORE: 5/14
  ```

  這是一個多語言模型，參數較小，但在處理中文與英文混合的財報查詢時展現較好。

- Model B：BAAI/bge-small-en-v1.5

  ```
  📊 FINAL SCORE: 1/14
  ```

  雖然在英文基準測試表現優異，但它是純英文優化模型。在你的測試環境中，面對中文提問時，檢索相關性大幅下降，導致後續節點無法運作。

- Reasoning
  - Cross-lingual Gap：
    由於問題包含中文（例如：「總營收」），MiniLM 作為多語言模型能正確將中文問題映射到英文財報片段。BGE-small-en 則因缺乏中文語義空間，導致檢索出的文檔大多被 grade_documents_node 標記為 ❌ Irrelevant，進而觸發無窮的 rewrite_node 循環直至 API 額度耗盡。
  - Embedding Precision：
    財務報表包含大量相似的數字與術語。在 MiniLM 下，即使檢索到了正確片段，系統仍有 FAIL（如 Tesla 研發費用），說明輕量級模型在區分「季度數據」與「全年總計」時的局限性。

- Trade-off
  模型並非越新、排名越高越好，須考慮問題語系與專業領域的適配。

Q2：Detailed comparison between LangGraph and LangChain.

特性	LangChain	LangGraph
結構模型	DAG（**有向無環圖**）：流程是線性的，難以回頭。	Cyclic Graph（**有向有環圖**）：支援循環與疊代。
狀態管理	**無狀態**（Stateless）：每一	**內置狀態**（State-aware）：全

	棒只傳遞前一棒的輸出。	域共享 AgentState。
控制權	**高層級抽象**：ReAct Agent 決定一切，像黑盒子。	**精細控制**：開發者定義節點與跳轉邏輯。
錯誤修復	依賴 LLM 自行修正，容易陷入死循環。	**強制修復**：可設計專門的 Rewrite 節點。
適用場景	簡單的問答、固定步驟的流水線。	複雜的財務分析、需要自我糾錯的 RAG。

- Cyclic Iteration

  LangChain 雖然有 ReAct 迴圈，但它是內建在 create_react_agent 裡的。如果 LLM 檢索到垃圾資訊，它可能會胡言亂語或直接放棄。

  LangGraph 增加 grade_documents 節點 。如果判定為 "no"，流程會強制導向 rewrite_query 。這種「不合格就重來」的顯式邏輯是 LangGraph 的核心優勢。

- State Awareness

  AgentState 記錄了 search_count。這在 LangChain 的傳統 Chain 中很難達成。你可以設定「如果重試超過 2 次，就強制進入生成階段」，避免 API 無止盡消耗。

- Intelligent Routing

  在 LangChain 中，你可能需要將所有工具塞給 LLM 。

  在 LangGraph 中，系統會先判斷是 "apple"、"tesla" 還是 "both" ，這減少了 LLM 在檢索時受到的干擾資訊，提高了 Context Precision。(Task B: Router)。

- 實作遇到的問題

  由於 LangGraph 將流程拆解成多節點，且引入 Cycles 與 Self-Correction 機制，API 的調用次數會呈現指數級增長。因此每次跑 evaluator 後面都會超過每日限額額度。

  ```
  ⚠️Grading failed: Error code: 429 - {'error': {'message': 'Rate limit
  ```

Q3：We used chunk_size=1000. If you add it or reduce it, how does it affect the ability to answer questions about large tables (like the Balance Sheet)? Explain the trade-off between "Context Precision" (small chunks) and "Context Completeness" (large chunks).

- large tables
  1. 增加 Chunk Size=2000
     能夠將整個資產負債表或主要部分封裝在單個片段中 。這確保了數據（數字）與其對應的行標籤（項目名稱）和列標籤（年份）不會被切斷 。

     結果：Agent 更容易回答「比較 2024 與 2023 總資產」這類需要跨行參考的問題，準確度提升。
  2. 減少 Chunk Size=500
     表格會擷取不完整。Agent 可能檢索到包含數字的片段，但該片段失去了頂部的「年份標題」或左側的「會計科目」 。
     結果：導致 LLM 因為缺乏上下文而產生幻覺，或是因無法確認年份而回答 "I don't know"。

- trade-off
  Context Precision：檢索到的內容中，「有用資訊」佔「總文字量」的比例，傾向 Small chunk。
  - Small chunk 能精確定位到特定的句子 。
  - 減少了與問題無關的噪音干擾，節省 LLM 的 Token 消耗 。
  - **風險**：容易因資訊破碎化而失去語義完整性 。

  Context Completeness：檢索到的內容是否包含回答問題所需的「全部背景」，傾向 Big chunk。

  - Big chunk 能保留報表的結構、定義與多個數據間的邏輯關係 。
  - 這對於需要「推理」或「總結」的財務問題（如分析趨勢）至關重要 。
  - **風險**：會引入大量無關文字（噪音），可能導致 LLM 注意力分散（Lost in the Middle 現象），且顯著增加 API 成本 。